

SmartCross: Railway Phatak Real-Time Status Web App

Project Proposal for UCS503P

Submitted to: Jhonsy Bansal

Thapar Institute of Engineering and Technology

Sandeep Yadav, CSED*

Moksh, CSED[†]

Gagandeep Singh, CSED[‡]

August 17, 2026

1 Higher-order goal

... secondary framing

This project aims to improve commuter safety and reduce time lost at unmanned and manned railway level crossings (“phataks”) by giving nearby residents and commuters real-time visibility into gate status, approaching trains, and alternate routes. While the broader goal is to reduce accidents and unnecessary waiting at railway crossings, the immediate engineering objective is to build a measurable, real-time web application, **SmartCross**, that fetches live train data, predicts gate closures, and pushes location-aware alerts to users near a phatak.

2 Time-to-value

... primary heuristic

- We will deliver functional increments quickly by first implementing live train data ingestion and a static map of nearby phataks, followed by prediction, real-time push alerts, and route suggestions in short development iterations.
- We will prioritize a working and measurable system over feature completeness by validating each module (data fetching, prediction, alerting, and the map UI) independently and gradually integrating advanced capabilities such as ETA prediction and alternate-route routing.
- Success will be defined by the ability to demonstrate real improvements in alert accuracy, alert lead-time, and reduced unnecessary waiting at phataks through controlled experiments and field-style testing.

3 Problem Statement

Railway level crossings are a frequent point of congestion, uncertainty, and risk for commuters travelling near railway lines. Gate closure timings are rarely known in advance, and commuters have no way of knowing whether a nearby phatak is open, about to close, or already closed until they physically reach it. Common challenges include:

- **No advance visibility of gate status:** Commuters do not know whether a phatak is open, closing soon, or closed until they arrive, leading to wasted time and unsafe last-moment crossing attempts.

*Roll No: 1024030312, Email: syadav1.be24thapar.edu

[†]Roll No: 1024030925, Email: mmoksh.be24thapar.edu

[‡]Roll No: 1024030314, Email: gsingh17.be24thapar.edu

- **Lack of train-approach awareness:** Without live train location data, users cannot anticipate when a gate will close or how long a closure will last.
- **No alternate route guidance:** When a phatak is closed, commuters are rarely offered a nearby alternate route, causing long queues and idle waiting.
- **Fragmented data sources:** Train running status, timetables, and phatak locations exist in separate systems with no unified, location-aware, real-time view for the end user.

Why this matters now (contextual relevance): As urban and semi-urban traffic increasingly crosses railway lines, giving commuters real-time, predictive, and location-aware information about phatak status can reduce congestion, prevent risky crossing behaviour, and improve daily commute reliability. Building SmartCross provides practical insight into real-time data pipelines, predictive ETA modelling, and location-based alerting systems.

4 Proposed Solution

... software proposition

4.1 Overview

The proposed solution is to build **SmartCross**, a web (and mobile-friendly) application that shows users nearby railway phataks, their live status, train-approaching alerts, and alternate routes. The system continuously ingests live train data, maps trains to the nearest phataks, predicts closure windows, and pushes real-time alerts to users located within a defined radius of a gate.

Unlike a static crossing-timetable display, SmartCross focuses on combining live data ingestion, prediction, and real-time push delivery to give commuters an actionable, up-to-the-minute view of the crossings around them. The system will consist of the following major components, organised into four layers as shown in Figure 1:

- **Data Sources:** A Train Status API supplies live train location, delay, next station, and speed, while Timetable Data supplies train schedules, routes, and frequency.
- **Backend Server:** Consists of a Data Fetcher that polls the Train Status API at regular intervals; a Data Processing & Mapping module that maps trains to tracks and the nearest phataks; a Prediction Engine that estimates train arrival time at each phatak and predicts closure time; a Gate Status Engine that determines gate state (Open, Closing Soon, Closed, Opening Soon); an Alert Engine that prepares alerts for users within a defined radius; and a Database that stores gates, trains, predictions, users, alerts, and history.
- **Real-Time Communication Layer:** A WebSocket Server pushes real-time status updates to connected clients, and a Notification Service sends push notifications to browser and mobile clients.
- **Web App (User):** A React-based frontend that shows nearby gates and their status, sends alerts when a phatak is closing or opening, displays estimated waiting time, suggests alternate routes, and works within a configurable radius (e.g. 1.5 km).

4.2 Core Workflow

The core workflow of SmartCross follows a complete real-time flow from a train running on the network to a user receiving an actionable alert:

1. A train is running, and its live location, speed, and delay are pulled from the Train Status API.

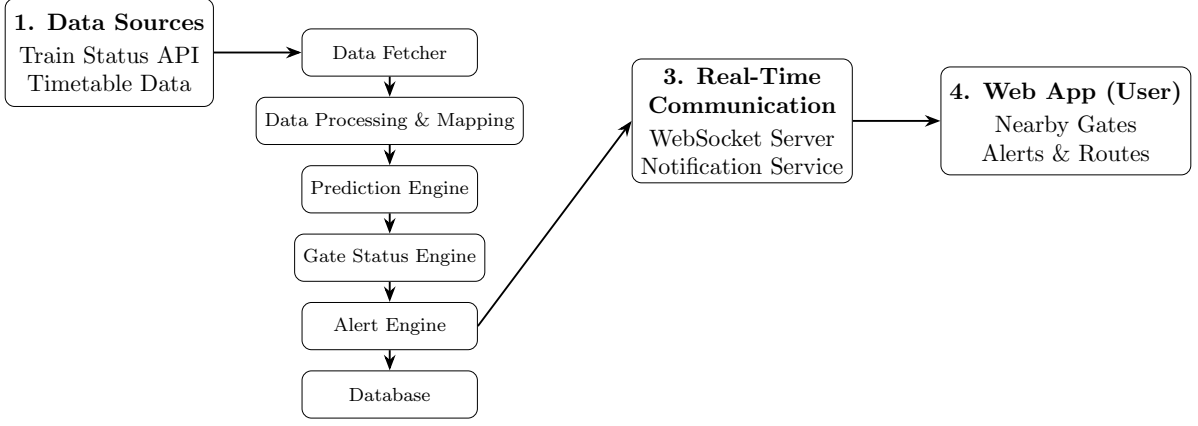


Figure 1: High-level architecture of SmartCross showing data sources, backend processing pipeline, real-time communication layer, and the user-facing web app.

2. The Data Processing & Mapping module maps the train to its track and the nearest phatak(s).
3. The Prediction Engine calculates the estimated time of arrival (ETA) of the train at the mapped phatak.
4. The Gate Status Engine predicts and updates the gate status (Open, Closing Soon, Closed, Opening Soon) based on the predicted ETA.
5. The Alert Engine prepares and sends an alert to users located within the defined radius of that phatak.
6. Users receive the real-time update through the web app or a push notification and can act accordingly, including viewing an alternate route.

4.3 Operational Constraints

The project is designed with the following operational constraints:

- **Controlled development environment:** SmartCross will initially operate using a limited set of phataks and a sample/public train-status data feed to allow controlled testing of prediction accuracy and alert delivery.
- **Lightweight implementation:** The system will focus on demonstrating live-data ingestion, prediction, and real-time alerting rather than depending on complex, production-scale railway infrastructure integrations.
- **Open and hosted technology stack:** The implementation will rely on React.js/Tailwind CSS on the frontend, Node.js/Express.js on the backend, MongoDB for storage, Socket.IO for real-time communication, and Google Maps/Mapbox API for maps and routing, deployed on AWS/Render.
- **Focus on correctness and timeliness:** The primary objective is to demonstrate accurate gate-status prediction and timely alert delivery rather than achieving nationwide, production-level coverage.
- **Radius-bound alerting:** Alerts are scoped to users within a configurable radius (e.g. 1.5 km) of a phatak; wider-area or city-scale rollout is future scope.

5 Solution Approach

... *engineering focus*

This project follows an engineering-focused approach where the emphasis is on designing, implementing, and evaluating a reliable real-time data pipeline and alerting system. The focus is on building a measurable, real-time web application rather than developing novel train-prediction algorithms. The system will be developed incrementally, with each major module tested independently before integration.

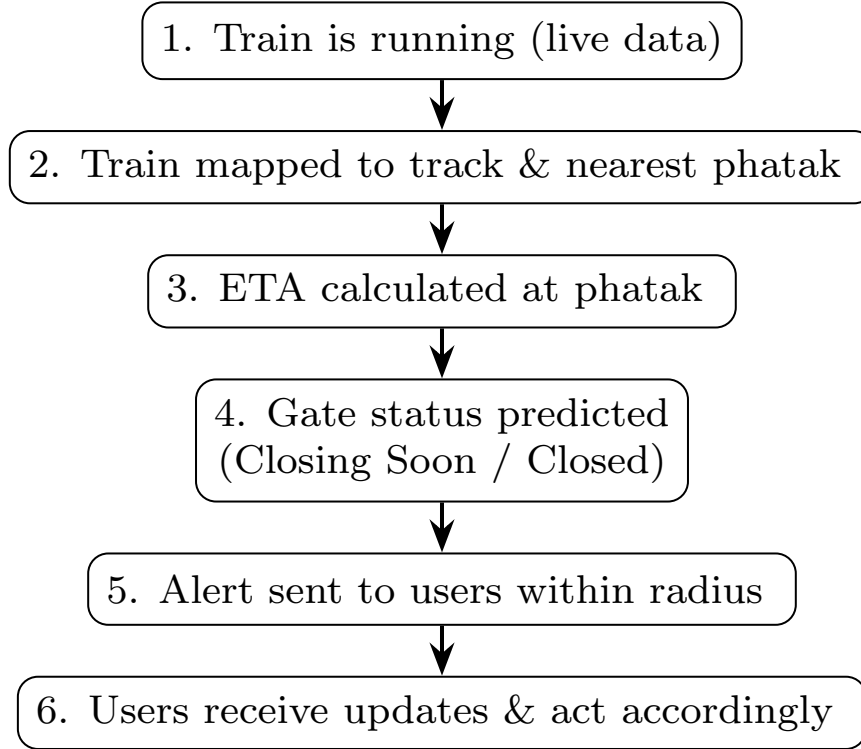


Figure 2: Real-time flow inside SmartCross from live train data to a user-facing alert.

5.1 Gate Status Prediction

The gate status logic will be implemented using a small set of well-defined states rather than a single binary open/closed flag.

- **State-based design:** A common gate-status model (Open, Closing Soon, Closed, Opening Soon) will be used consistently across the Prediction Engine, Gate Status Engine, and the frontend UI.
- **ETA-driven predictions:** The Prediction Engine estimates train arrival time at each mapped phatak using live speed, distance, and historical timetable data, and predicts the expected closure window.
- **Configurable alert radius:** The radius within which users receive alerts for a given phatak will be controlled through configuration, allowing tuning without modifying core logic.

5.2 System Architecture

SmartCross will follow a modular architecture where each responsibility is separated into independent components, as shown in Figure 1.

- **Data Fetcher:** Polls the Train Status API at regular intervals to keep live location, delay, and speed data current.
- **Data Processing & Mapping:** Maps live trains to tracks and to the nearest phataks using timetable and route data.
- **Prediction Engine:** Estimates train arrival time at each phatak and predicts closure time.
- **Gate Status Engine:** Determines and updates gate status (Open, Closing Soon, Closed, Opening Soon).
- **Alert Engine:** Prepares and dispatches alerts for users within a defined radius of a gate.
- **Real-Time Communication Layer:** A WebSocket Server pushes live updates, and a Notification Service delivers push notifications to browser and mobile clients.

5.3 Concurrency and Real-Time Updates

Since SmartCross must continuously ingest live train data while simultaneously serving many connected users, concurrency and real-time delivery are core implementation considerations.

- Live data polling from the Train Status API will run as an independent background process on a fixed interval.
- Gate status recalculation and alert preparation will run independently of client connections so that a slow client cannot block data processing.
- Shared state such as gate status, predicted ETAs, and active user sessions will be protected using appropriate synchronization mechanisms and stored centrally in the database.
- WebSocket connections will broadcast only the updates relevant to users within a given phatak's radius, to keep the real-time channel efficient.

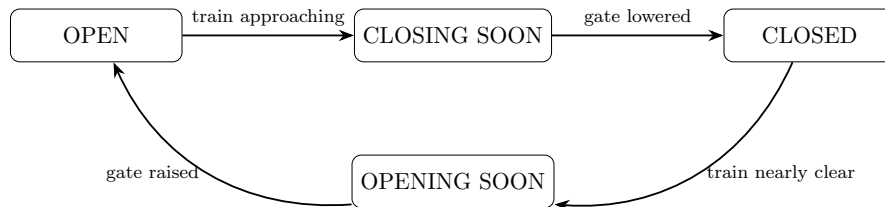


Figure 3: Gate status state transition used by SmartCross's Gate Status Engine.

5.4 Testing and Performance Evaluation

The system will be validated through controlled experiments that measure both prediction accuracy and real-time delivery performance.

- Test individual components such as train-to-phatak mapping, ETA prediction, gate-status transitions, and alert dispatch independently.
- Perform integration testing using a sample of phataks connected to live or simulated train-status data through SmartCross.
- Evaluate system behaviour during data feed delays, missing train data, and sudden bursts of nearby users.
- Measure performance indicators such as prediction accuracy, alert lead-time, notification delivery latency, and WebSocket update latency.

5.5 CI/CD and Incremental Delivery

Development will follow an incremental delivery approach to ensure that each stage produces a working system component.

- Maintain version control using Git and GitHub with regular commits for each implemented module.
- Use automated testing to verify system correctness after changes.
- Gradually integrate advanced features such as ETA-based prediction, push notifications, alternate-route suggestions, and history/analytics after the core data-fetching and mapping pipeline is stable.

6 Evaluation Criterion

...measurable and attributable

The success of SmartCross will be evaluated using measurable prediction accuracy, alert timeliness, and usability metrics directly related to the core system objectives. The evaluation will focus on how effectively the system predicts gate closures and delivers timely, location-relevant alerts.

6.1 Primary evaluation metric

Alert timeliness and accuracy: The primary evaluation metric will measure how effectively SmartCross predicts gate status changes and delivers alerts to nearby users before a phatak closes.

- **Target:** Demonstrate that “Closing Soon” alerts are delivered with a measurable positive lead-time before actual gate closure, with a low false-alert rate.
- **Attribution:** Measured by comparing predicted closure times against actual/simulated closure times, and by recording notification delivery latency for users within the alert radius.

6.2 Secondary evaluation metrics

- **Mapping accuracy:** Measure how correctly live trains are mapped to the nearest track and phatak under varying train density.
- **Gate-status transition correctness:** Evaluate how accurately and how quickly the Gate Status Engine transitions between Open, Closing Soon, Closed, and Opening Soon.
- **Real-time delivery latency:** Measure the time between a gate-status change and the corresponding update reaching the WebSocket-connected client and the Notification Service.
- **Route suggestion usefulness:** Assess whether the recommended alternate route, shown when a phatak is closed, offers a meaningfully shorter or faster path than waiting.
- **Observability quality:** Assess whether the stored history of gates, trains, predictions, and alerts accurately supports analytics and later debugging.

6.3 Validation plan

- Deploy SmartCross against a sample set of phataks using a live or simulated Train Status API feed.
- Compare predicted ETAs and gate-status transitions against actual/simulated train movement to measure prediction accuracy.

- Perform controlled experiments by simulating multiple users within and outside a phatak's alert radius to validate radius-bound alert delivery.
- Record system metrics such as alert lead-time, notification latency, WebSocket update latency, and prediction error to validate the effectiveness of the implementation.

7 Scalability

...with foresight

SmartCross is designed with scalability considerations at both the data-ingestion and system-architecture levels. Although the initial implementation focuses on a limited set of phataks and a single backend instance, the modular design allows future expansion.

- **Phatak scalability:** The database schema for gates, trains, and predictions is designed to support additional phataks being added through configuration and data-mapping rather than code changes.
- **Prediction scalability:** The Prediction Engine's interface allows more sophisticated ETA models (e.g. incorporating historical delay patterns) to be introduced without changing the surrounding pipeline.
- **User scalability:** The WebSocket Server and Notification Service are designed around radius-scoped broadcasting, allowing the system to serve many concurrently connected users without pushing irrelevant updates.
- **Future distributed scaling:** A future version could support multiple regional backend instances and a distributed database. However, this introduces additional challenges such as cross-region data consistency and centralized monitoring, which are outside the scope of the current implementation.

8 Engineering Available Heuristic

The project follows established engineering practices and existing real-time system concepts rather than attempting to develop new train-prediction algorithms. The objective is to understand, implement, and evaluate proven techniques for live-data ingestion, prediction, and real-time delivery used in real-world systems.

- **Existing real-time patterns:** Well-known techniques such as periodic polling, WebSocket-based push updates, and radius-based geofencing will be implemented and combined instead of designing new distributed protocols.
- **Incremental development:** The system will be developed in stages, starting with basic data fetching and gate-status display and gradually adding prediction, real-time alerts, and route suggestions.
- **Modular engineering approach:** Components such as the Data Fetcher, Data Processing & Mapping, Prediction Engine, Gate Status Engine, Alert Engine, and Notification Service will remain independent to simplify testing and future improvements.
- **Practical validation:** System behaviour will be evaluated through controlled experiments and measurable benchmarks rather than theoretical assumptions.

9 Project scope and deliverables

...iteration-friendly

The project scope focuses on developing a functional real-time phatak status web app with live gate tracking, predictive alerts, and route suggestions. Development will follow an incremental approach, where the core data pipeline and status display are implemented first, followed by prediction, real-time alerting, and analytics features.

9.1 Initial deliverable

The first iteration will focus on establishing the core functionality of SmartCross:

- **Basic data ingestion:** Implement the Data Fetcher to pull live train location, delay, next station, and speed from the Train Status API at regular intervals.
- **Phatak and train mapping:** Create the Data Processing & Mapping module and database schema to map trains to tracks and nearest phataks.
- **Basic gate status display:** Implement the Gate Status Engine and a web UI showing nearby gates and their current status (Open, Closing Soon, Closed, Opening Soon).
- **Configurable phatak setup:** Support defining phataks, their coordinates, and basic system settings through a configuration file/database seed.
- **Unit testing and validation:** Test core components such as data fetching, mapping, and gate-status determination.
- **Version control and development workflow:** Maintain the project repository with structured commits and incremental feature integration.

9.2 Subsequent deliverables

Later iterations will extend the system with prediction, real-time alerting, and analytics capabilities:

- **Prediction Engine:** Implement ETA estimation and closure-time prediction, and evaluate accuracy under different train densities.
- **Real-time communication:** Add the WebSocket Server and Notification Service to push live gate-status updates and alerts to browser/mobile clients.
- **Radius-based alerting:** Implement the Alert Engine to prepare and dispatch alerts for users within a configurable radius of a phatak.
- **Alternate route suggestions:** Integrate Google Maps/Mapbox API to recommend alternate routes when a phatak is closed.
- **Estimated waiting time:** Surface predicted waiting time on the web app based on the Prediction Engine's output.
- **History and analytics:** Store gate, train, prediction, and alert history in the database and expose basic analytics.
- **Benchmarking and evaluation:** Conduct experiments measuring prediction accuracy, alert lead-time, notification latency, and WebSocket update latency.

10 Risks and Mitigations

- **Live data feed unavailability or inaccuracy:** The Train Status API may have gaps, delays, or inaccuracies that reduce prediction quality. **Mitigation:** Use a fallback timetable-based estimate when live data is stale or missing, and clearly surface prediction confidence to users.
- **Concurrency and synchronization issues:** Simultaneous data polling, prediction updates, and many connected WebSocket clients may introduce race conditions or inconsistent gate-status states. **Mitigation:** Design shared components carefully and use synchronization mechanisms such as locks or atomic database updates. Test concurrent behaviour early using controlled workloads.

- **Scope expansion:** Implementing a complete, nationwide phatak-monitoring platform involves several advanced features beyond the semester timeline. **Mitigation:** Prioritize the core MVP consisting of data ingestion, mapping, gate-status prediction, and radius-based alerting for a limited set of phataks. City-scale or nationwide rollout will remain future scope.
- **Prediction accuracy variability:** ETA and closure-time predictions may vary depending on train speed changes, delays, and incomplete data. **Mitigation:** Keep prediction logic simple and transparent initially, validate against known train movements, and iterate based on measured error rates.
- **False or missed alerts:** Incorrect gate-status predictions could lead to false “Closing Soon” alerts or missed closures, reducing user trust. **Mitigation:** Evaluate alert precision/recall carefully before enabling automatic push notifications, and allow a manual/community-reported override for known-incorrect statuses.
- **Route-suggestion feature affecting core progress:** Building maps and alternate-route integration may consume time needed for the core data pipeline and alerting. **Mitigation:** Develop route suggestions only after the core status-tracking and alerting components are stable, using already available Maps APIs.
- **Security and privacy limitations:** Location-based alerting involves user location data that requires careful handling. **Mitigation:** Clearly define the project as an educational implementation, minimize stored location data, and avoid persisting precise user location beyond what is needed for radius-based alerting.

11 Summary

This project proposes the development of SmartCross, a real-time railway phatak status web application designed to give commuters visibility into nearby gate status, train-approach alerts, and alternate routes. The system focuses on live data ingestion, predictive gate-status modelling, radius-based real-time alerting, and route suggestions through a modular engineering approach.

The project emphasizes measurable evaluation through controlled experiments involving prediction accuracy, alert lead-time, notification latency, and real-time update delivery. By implementing established real-time system concepts such as periodic data polling, WebSocket-based push updates, and geofenced alerting, SmartCross aims to provide practical understanding of how modern location-aware, real-time systems handle prediction, reliability, and scalability challenges.

The project remains focused on engineering implementation and evaluation rather than developing novel train-prediction algorithms. Its contribution lies in building a functional, measurable, and extensible application that demonstrates the principles behind real-world, real-time public-safety and commuter-information systems.